

AI Meeting Memory Vault

Project Description

AI Meeting Memory Vault is a backend-based intelligent meeting management system developed using Node.js, Express.js, MongoDB, and Google Gemini AI. The system helps organizations and teams store meeting transcripts, generate AI-powered summaries, search historical meeting records, analyze meeting data, and interact with meeting knowledge through an AI chat interface.

The project eliminates the difficulty of manually reviewing lengthy meeting notes by automatically generating concise summaries, extracting key information, and making past meeting data easily searchable. It serves as a centralized repository for preserving organizational knowledge and meeting history.

Project Objective

The primary objective of this project is to build an intelligent meeting repository that can:

- Store meeting transcripts and related information.
 - Generate AI-powered summaries from meeting discussions.
 - Provide quick access to historical meeting records.
 - Enable intelligent search across meeting data.
 - Offer analytical insights about meetings.
 - Allow users to interact with stored meeting knowledge through AI-powered conversations.
 - Demonstrate the practical integration of Artificial Intelligence with modern database systems.
-

Project Scope

The scope of this project includes:

Meeting Management

- Create new meeting records.
- Retrieve meeting information.
- Update existing meeting details.
- Delete obsolete meeting records.

AI-Powered Features

- Automatic meeting summarization using Gemini AI.
- Generation of concise meeting overviews.
- Extraction of action items and decisions.
- AI-based question answering from stored meeting history.

Search Functionality

- Full-text search across meeting transcripts.
- Search based on keywords and topics.
- Quick retrieval of relevant meeting information.

Analytics

- Meeting statistics dashboard.
- Meeting count analysis.
- Action item tracking.
- Decision tracking.

File Processing

- Upload transcript files.
- Automatic processing of uploaded content.
- Storage of transcript data in MongoDB.

Data Management

- Centralized storage of meeting records.

- Efficient retrieval of historical information.
 - Support for pagination and filtering.
-

Problem Statement

Organizations conduct numerous meetings every day, generating large amounts of information. Important decisions, action items, and discussions often become difficult to locate later. Traditional note-taking methods make searching and retrieving relevant information time-consuming.

AI Meeting Memory Vault addresses this problem by creating a centralized AI-powered repository where meeting data can be stored, summarized, searched, and analyzed efficiently.

Technology Stack

Backend Technologies

Node.js

Used as the JavaScript runtime environment for executing server-side code.

Express.js

Used to build RESTful APIs and manage backend routing.

MongoDB

Used as the NoSQL database for storing meeting information and AI-generated summaries.

Mongoose

Used as an Object Data Modeling (ODM) library for MongoDB.

Multer

Used for handling transcript file uploads.

Dotenv

Used for environment variable management.

Artificial Intelligence

Google Gemini AI

Used to:

- Generate meeting summaries.
- Analyze meeting transcripts.
- Answer questions from meeting history.
- Extract important meeting information.

API Testing & Documentation

Postman

Used for API development and testing.

Swagger UI

Used for API documentation and endpoint visualization.

Database Design

The project uses a MongoDB collection named:

meetings

Key fields stored in the database:

- Title
- Transcript
- Summary Overview
- Action Items
- Decisions
- Tags

- Participants
 - Meeting Date
 - Created At
 - Updated At
-

Key Features

1. Meeting Management

Provides complete CRUD functionality:

- Create Meeting
 - Read Meeting
 - Update Meeting
 - Delete Meeting
-

2. Transcript Upload

Users can upload meeting transcript files through API endpoints.

Features:

- File upload support.
 - Automatic transcript extraction.
 - Database storage.
-

3. AI Summary Generation

The system uses Gemini AI to analyze transcripts and generate:

- Meeting overview.
- Key action items.
- Important decisions.

4. Full-Text Search

Users can search historical meetings using keywords.

Examples:

- MongoDB
- Backend
- AI
- Deployment

MongoDB Text Indexing enables efficient search operations.

5. Dashboard Analytics

The dashboard provides:

- Total meetings count.
- Total action items.
- Total decisions.
- Aggregated statistics.

MongoDB Aggregation Pipelines are used for analytics.

6. AI Meeting Chat

Users can ask natural language questions such as:

- What decisions were made regarding MongoDB?
- List all action items from previous meetings.
- What discussions happened about backend development?

The system retrieves relevant meeting information and generates responses using Gemini AI.

7. Pagination and Filtering

Supports:

- Paginated data retrieval.
- Tag-based filtering.
- Participant-based filtering.

This improves performance and scalability.

System Workflow

User Uploads Transcript



Transcript Stored in MongoDB



Gemini AI Processes Transcript



Summary Generated



Summary Stored in Database



Search / Analytics / AI Chat Available

API Modules

Meeting APIs

- Create Meeting
- Get All Meetings
- Get Meeting By ID

- Update Meeting
- Delete Meeting

Search APIs

- Search Meetings

Analytics APIs

- Dashboard Statistics

AI APIs

- Generate Summary
- Upload Transcript
- AI Chat

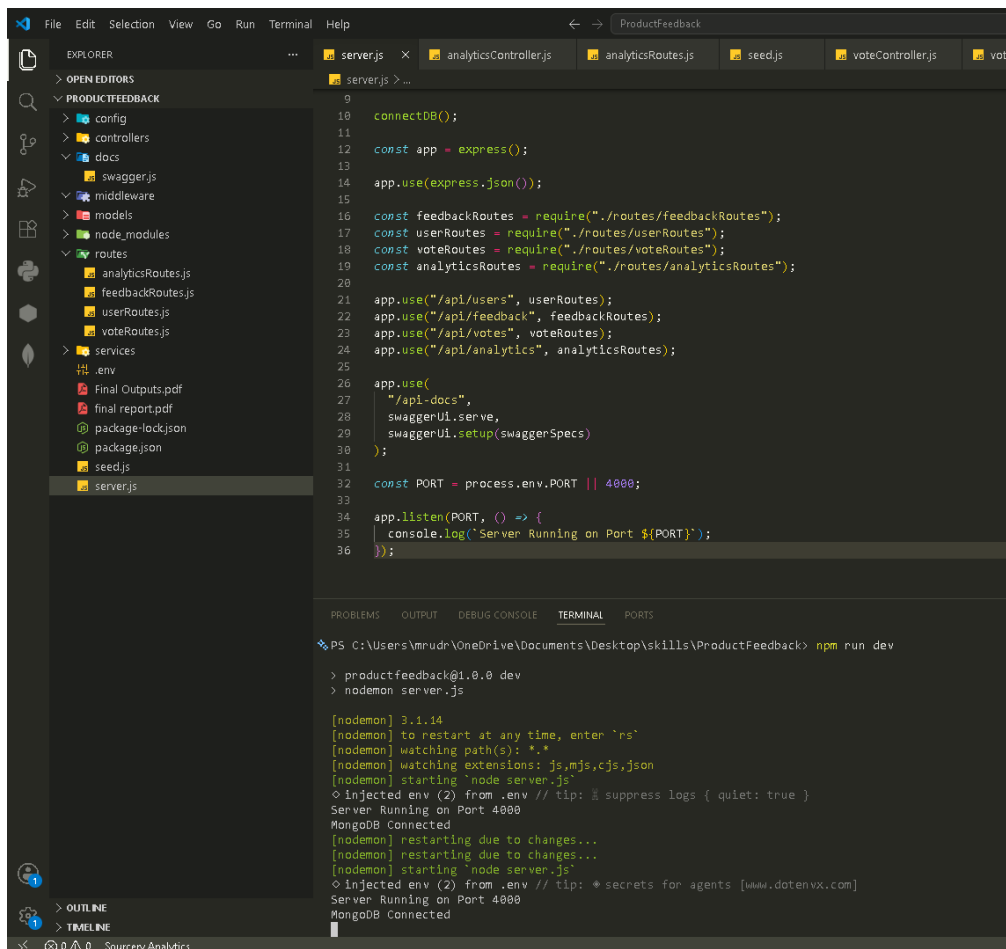
Learning Outcomes

Through this project, the following concepts were implemented and learned:

- REST API Development
- Express.js Routing
- MongoDB CRUD Operations
- Mongoose Schema Design
- File Upload Handling
- Environment Variable Management
- AI Integration with Gemini
- Text Search in MongoDB
- Aggregation Pipelines
- Backend Project Structuring
- API Documentation using Swagger
- Postman API Testing

Conclusion

AI Meeting Memory Vault is an intelligent backend application that combines MongoDB, Express.js, and Google Gemini AI to create a centralized repository for meeting knowledge. The project demonstrates practical implementation of Artificial Intelligence in real-world business workflows by automating meeting summarization, enabling efficient information retrieval, and providing analytical insights from stored meeting data.



```
> aimeetingmemorevault@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (4) from .env // tip: ◇ secrets for agents [www.dotenvx.com]
Server Running On Port 7000
MongoDB Connected Successfully
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (4) from .env // tip: ~ auth for agents [www.vestauth.com]
Server Running On Port 7000
MongoDB Connected Successfully
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (4) from .env // tip: % override existing { override: true }
Server Running On Port 7000
MongoDB Connected Successfully
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (4) from .env // tip: % enable debugging { debug: true }
Server Running On Port 7000
MongoDB Connected Successfully
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (4) from .env // tip: ◇ encrypted .env [www.dotenvx.com]
Server Running On Port 7000
MongoDB Connected Successfully
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (4) from .env // tip: % enable debugging { debug: true }
Server Running On Port 7000
MongoDB Connected Successfully
```

Create Meeting

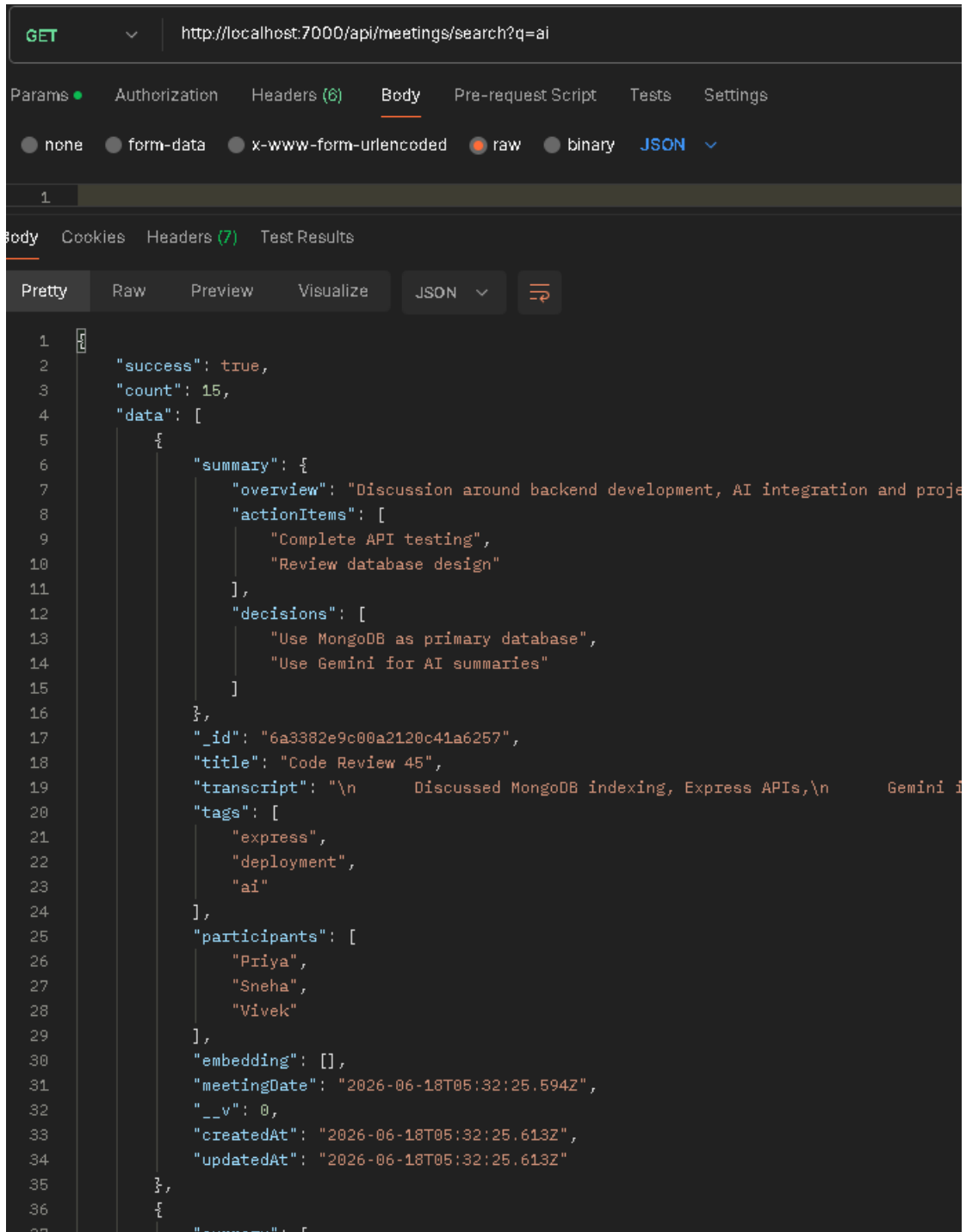
The screenshot displays a REST client interface with a POST request to `http://localhost:7000/api/meetings`. The request body is a JSON object with the following fields:

```
1 {
2   "title": "Sprint Planning Meeting",
3   "transcript": "Discussed backend architecture and MongoDB indexing.",
4   "participants": ["Rudraksh", "Aman"],
5   "tags": ["mongodb", "backend"]
6 }
```

The response is shown in the 'Body' tab, formatted as JSON. It includes a success flag, the request data, a summary object, tags, participants, an embedding array, and various identifiers and timestamps.

```
1 {
2   "success": true,
3   "data": {
4     "title": "Sprint Planning Meeting",
5     "transcript": "Discussed backend architecture and MongoDB indexing.",
6     "summary": {
7       "overview": "",
8       "actionItems": [],
9       "decisions": []
10    },
11    "tags": [
12      "mongodb",
13      "backend"
14    ],
15    "participants": [
16      "Rudraksh",
17      "Aman"
18    ],
19    "embedding": [],
20    "_id": "6a33f6a5b6a0e485b46ac1cf",
21    "meetingDate": "2026-06-18T13:46:13.328Z",
22    "createdAt": "2026-06-18T13:46:13.340Z",
23    "updatedAt": "2026-06-18T13:46:13.340Z",
24    "__v": 0
25  }
26 }
```

Search AI



```
GET http://localhost:7000/api/meetings/search?q=ai

Params Authorization Headers (6) Body Pre-request Script Tests Settings
none form-data x-www-form-urlencoded raw binary JSON

1
body Cookies Headers (7) Test Results
Pretty Raw Preview Visualize JSON

1
2  "success": true,
3  "count": 15,
4  "data": [
5    {
6      "summary": {
7        "overview": "Discussion around backend development, AI integration and proje",
8        "actionItems": [
9          "Complete API testing",
10         "Review database design"
11       ],
12       "decisions": [
13         "Use MongoDB as primary database",
14         "Use Gemini for AI summaries"
15       ]
16     },
17     "_id": "6a3382e9c00a2120c41a6257",
18     "title": "Code Review 45",
19     "transcript": "\n      Discussed MongoDB indexing, Express APIs,\n      Gemini i",
20     "tags": [
21       "express",
22       "deployment",
23       "ai"
24     ],
25     "participants": [
26       "Priya",
27       "Sneha",
28       "Vivek"
29     ],
30     "embedding": [],
31     "meetingDate": "2026-06-18T05:32:25.594Z",
32     "__v": 0,
33     "createdAt": "2026-06-18T05:32:25.613Z",
34     "updatedAt": "2026-06-18T05:32:25.613Z"
35   },
36   {
37     "summary": {
```

Dashboard Analytics

The screenshot shows a REST client interface with a GET request to `http://localhost:7000/api/meetings/dashboard`. The response is displayed in the 'Body' tab, showing a JSON object with the following structure:

```
1  {
2    "success": true,
3    "data": {
4      "totalMeetings": 51,
5      "totalActionItems": 100,
6      "totalDecisions": 100
7    }
8  }
```

The interface includes tabs for Params, Authorization, Headers (6), Body, Pre-request Script, Tests, and Settings. The 'Body' tab is active, showing the response in JSON format. The response is a JSON object with a 'success' property set to true and a 'data' property containing an object with three metrics: totalMeetings (51), totalActionItems (100), and totalDecisions (100).

Pagination Page 1

The screenshot shows a REST client interface with a GET request to `http://localhost:7000/api/meetings?page=1&limit=5`. The response is displayed in the 'Body' tab, showing a JSON object with the following structure:

```
1 {
2   "success": true,
3   "page": 1,
4   "limit": 5,
5   "totalMeetings": 51,
6   "totalPages": 11,
7   "count": 5,
8   "data": [
9     {
10      "summary": {
11        "overview": "",
12        "actionItems": [],
13        "decisions": []
14      },
15      "_id": "6a33f6a5b6a0e485b46ac1cf",
16      "title": "Updated Sprint Planning Meeting",
17      "transcript": "Discussed backend architecture and MongoDB indexing.",
18      "tags": [
19        "mongodb",
20        "backend"
21      ],
22      "participants": [
23        "Rudraksh",
24        "Aman"
25      ],
26      "embedding": [],
27      "meetingDate": "2026-06-18T13:46:13.328Z",
28      "createdAt": "2026-06-18T13:46:13.340Z",
29      "updatedAt": "2026-06-18T13:48:50.851Z",
30      "__v": 0
31    },
32    {
33      "summary": {
34        "overview": "Discussion around backend development, AI integration and project planning.",
35        "actionItems": [
36          "Complete API testing",
37          "Review database design"
38        ]
39      }
40    }
41  ]
42 }
```

Generate AI Summary

POST

http://localhost:7000/api/meetings/6a3382e9c00a2120c41a622b/summary

Send

Params

Authorization

Headers (7)

Body

Pre-request Script

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

JSON

1

Body

Cookies

Headers (7)

Test Results

Status: 200 OK

Time: 2.52 s

Size: 982 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

```
1  {
2    "success": true,
3    "message": "AI summary generated successfully",
4    "data": {
5      "summary": {
6        "overview": "The meeting covered several topics including MongoDB indexing, Express APIs, Gemini integration, dashboard analytics, deployment planning, and overall project progress.",
7        "actionItems": [],
8        "decisions": []
9      },
10     "id": "6a3382e9c00a2120c41a622b",
11     "title": "Code Review 1",
12     "transcript": "\n      Discussed MongoDB indexing, Express APIs,\n      Gemini integration, dashboard analytics,\n      deployment planning and project progress.\n      ",
13     "tags": [
14       "database",
15       "nodejs",
16       "gemini"
17     ],
18     "participants": [
19       "Vivek",
20       "Sneha",
21       "Karan"
22     ],
23     "embedding": [],
24     "meetingDate": "2026-06-18T06:32:25.694Z",
25     "_v": 0,
26     "createdAt": "2026-06-18T06:32:25.611Z",
27     "updatedAt": "2026-06-18T13:56:38.092Z"
28   }
29 }
```

AI Chat

POST

http://localhost:7000/api/meetings/chat

Params

Authorization

Headers (8)

Body

Pre-request Script

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

JSON

1

2

3

"question": "what decisions were made about MongoDB?"

Body

Cookies

Headers (7)

Test Results

Status: 200 OK

Time: 3.27 s

Size: 480 B

Pretty

Raw

Preview

Visualize

JSON

1

2

3

4

5

"success": true,

"question": "What decisions were made about MongoDB?",

"answer": "Based on the provided meeting data, specific decisions made about MongoDB are not explicitly stated. The meetings indicate that \"MongoDB indexing\" was discussed."